

Sistema IoT de Vigilancia

Carlos Nicolas Morales Cuellar, Jose Ramos Ramos Calzada , Ronald Enrique Rojas Crisostomo

Universidad Nacional Mayor de San Marcos

Lma, Perú

carlosnicolasmorales2004@gmail.com

jose.ramos4b@gmail.com

ronald.rojasc@unmsm.edu.pe

Resumen— Este artículo presenta el diseño, implementación y evaluación de un sistema inteligente de vigilancia y alerta remota orientado a mitigar problemas de inseguridad ciudadana mediante tecnologías de Internet de las Cosas (IoT) y Edge Computing. A diferencia de las cámaras comerciales que operan como "cajas negras", esta arquitectura open-source de "caja blanca" garantiza la soberanía de los datos al procesar las activaciones de movimiento localmente mediante sensores infrarrojos pasivos (PIR) integrados a nodos de bajo costo (Raspberry Pi Zero 2W y ESP32-CAM). Un servidor local desarrollado en Flask actúa como middleware, gestionando la comunicación LAN de forma stateless y comunicándose asincrónicamente con la API de Telegram vía TLS 1.3 para la emisión de alertas. El diseño fue validado en entornos físicos y simulados (GNS3), evaluando métricas de latencia, throughput y tolerancia a la congestión. Los resultados demuestran una alta eficiencia de ancho de banda, al requerir transferencias únicamente ante eventos específicos, evitando la saturación del enlace de subida típico de los sistemas de videovigilancia tradicionales en la nube.

Palabras clave— Internet de las Cosas (IoT), Edge Computing, Redes Inalámbricas, Inspección Profunda de Paquetes (DPI), Seguridad Ciudadana.

Abstract— This paper presents the design, implementation, and evaluation of an intelligent remote surveillance and alert system aimed at mitigating public insecurity issues through Internet of Things (IoT) and Edge Computing technologies. Unlike commercial cameras that operate as "black boxes," this open-source "white-box" architecture ensures data sovereignty by processing motion activations locally using passive infrared (PIR) sensors integrated into low-cost nodes (Raspberry Pi Zero 2W and ESP32-CAM). A local server developed in Flask acts as middleware, managing LAN communication in a stateless manner and asynchronously communicating with the Telegram API via TLS 1.3 to issue alerts. The design was validated in both physical and simulated environments (GNS3), evaluating latency, throughput, and congestion tolerance metrics. The results demonstrate high bandwidth efficiency by requiring data transfers only during specific events, avoiding the uplink saturation typical of traditional cloud-based video surveillance systems.

Keywords— Internet of Things (IoT), Edge Computing, Wireless Networks, Deep Packet Inspection (DPI), Public Security.

I. INTRODUCCIÓN

A. Problema

La inseguridad ciudadana en el Perú, evidenciada en el

incremento de delitos urbanos, exige el despliegue de soluciones tecnológicas de monitoreo que sean accesibles, eficientes y auditables. En el mercado actual, las soluciones de videovigilancia convencionales operan bajo un modelo cerrado de "caja negra". Este esquema impide a los administradores de red tener visibilidad sobre la telemetría, identificar puertos expuestos o controlar el destino de los metadatos exportados, además de saturar el ancho de banda de subida (uplink) al transmitir flujos de video constantes hacia nubes propietarias de terceros.

Frente a estas limitaciones, este artículo presenta el diseño y la evaluación de un Sistema Inteligente de Vigilancia y Alerta Remota concebido esencialmente como un proyecto de código abierto (open-source) y de arquitectura "caja blanca". Alojado en repositorios públicos como GitHub, la propuesta de valor central no radica en competir como un producto final hermético, sino en establecer una plataforma base altamente colaborativa y documentada. El diseño se fundamenta en el desacoplamiento de servicios mediante un servidor middleware local (Flask) y nodos de procesamiento en el borde (Edge Computing) basados en hardware de bajo costo (Raspberry Pi Zero 2W y ESP32-CAM).

Al procesar los disparadores de movimiento de forma estrictamente local mediante sensores infrarrojos pasivos (PIR), el sistema optimiza drásticamente los recursos de red, generando tráfico únicamente ante eventos confirmados, aunque también el usuario puede ejecutar comandos para interactuar con la cámara. La naturaleza modular de esta arquitectura descentralizada garantiza una escalabilidad lineal y orgánica. Esto permite que cualquier estudiante de telecomunicaciones o desarrollador de la comunidad global pueda clonar el repositorio, adaptar la topología a entornos de mayor envergadura y añadir funciones personalizadas según las necesidades de sus propios proyectos —como el despliegue de scripts de *Machine Learning* para reconocimiento facial en el servidor local o la integración de pasarelas de alerta alternativas (WhatsApp en vez de Telegram, paneles de control personalizados o sistemas SIEM)— sin alterar el *firmware* central de los nodos sensores ni depender de suscripciones comerciales en la nube.

B. Preguntas de investigación

- ¿De qué manera una arquitectura IoT open-source y descentralizada permite mitigar el impacto de la saturación del canal de subida en redes WLAN residenciales?
- ¿Cómo influye la modularidad de un servidor middleware local en la escalabilidad y adaptabilidad de funciones desarrolladas de forma colaborativa en GitHub?
- ¿En qué medida el enfoque de "caja blanca" y la observabilidad de red mediante inspección profunda de paquetes (DPI) optimizan el diagnóstico de fallos frente

a soluciones comerciales propietarias?

C. Objetivos

- Objetivo General: Diseñar e implementar un sistema inteligente de vigilancia IoT de código abierto basado en Edge Computing que constituya una plataforma colaborativa, escalable y eficiente en el uso del ancho de banda de red.
- Objetivos Específicos:
 - Desplegar una topología modular con nodos en el borde (Raspberry Pi y ESP32-CAM) para descentralizar el procesamiento de capturas multimedia y asegurar la escalabilidad del hardware.
 - Desarrollar un servidor intermedio local en Flask que actúe como un puente flexible, facilitando a la comunidad la adición de nuevas funciones y la integración de APIs externas.
 - Validar el rendimiento y la transparencia del sistema mediante el análisis minucioso de protocolos de red y pruebas de estrés ante escenarios de congestión y fallos de enlace.

D. Alcance

El alcance de este trabajo comprende el diseño, implementación y documentación detallada de la arquitectura física y lógica en una red de área local inalámbrica (WLAN 802.11n en 2.4 GHz). Esto abarca la programación del software de control en los nodos del borde para la captura multimedia, la configuración del servidor intermedio basado en Flask y el establecimiento del canal de alertas cifrado hacia la API de Telegram. Asimismo, el proceso de evaluación se centra exclusivamente en la caracterización y auditoría del tráfico real en un entorno físico de prueba mediante inspección profunda de paquetes (DPI), analizando métricas de rendimiento como latencia basal, throughput, retransmisiones y el comportamiento de los protocolos de transporte ante eventos de activación y saturación inducida directamente sobre el hardware local.

D. Limitaciones

Al ser un proyecto que delega alertas remotas hacia el exterior, la velocidad de notificación final depende de la estabilidad y el ancho de banda del proveedor de servicios de Internet (ISP) local para alcanzar la API de Telegram. Asimismo, la capacidad de procesamiento simultáneo de hilos en el nodo esclavo (ESP32-CAM) está restringida por sus recursos de hardware nativos, limitando la ejecución de análisis multimedia masivos dentro del microcontrolador.

II. ESTADO DEL ARTE

El desarrollo de sistemas de videovigilancia ha experimentado una rápida evolución hacia arquitecturas integradas con el Internet de las Cosas (IoT). Diversos autores han propuesto sistemas inteligentes que utilizan sensores de bajo costo para mitigar la inseguridad y prevenir pérdidas patrimoniales en tiempo real [1]. Sin embargo, la mayoría de las alternativas tradicionales dependen de la transmisión continua de video hacia servidores de terceros en la nube (Cloud Computing). Según estudios sobre aplicaciones de transmisión en tiempo real, este modelo centralizado genera cuellos de botella severos de latencia y satura el canal de subida (uplink), haciéndolo ineficiente para entornos residenciales con recursos de red limitados [2].

Para resolver este problema de saturación, la literatura académica más reciente propone la transición hacia el Edge Computing (Computación en el Borde). Investigaciones centradas en optimización de redes demuestran que procesar los datos de los sensores localmente y transmitir a la nube únicamente la información esencial ante un evento confirmado reduce el consumo de ancho de banda hasta en un 25% [3]. Asimismo, frameworks recientes de Edge Computing confirman que esta descentralización no solo disminuye la latencia, sino que incrementa la seguridad al limitar la transmisión de información sensible fuera de la red local [4]. El diseño de nuestro sistema adopta plenamente este paradigma, delegando la activación, captura y compresión a los nodos locales (Raspberry Pi Zero 2W y ESP32-CAM).

Desde la perspectiva de la gestión de redes y la ciberseguridad, las cámaras IP comerciales operan bajo un modelo hermético. Investigaciones enfocadas en el análisis de tráfico IoT advierten que esta falta de transparencia impide a los usuarios auditar los flujos de paquetes, generando riesgos de privacidad y exfiltración de telemetría no deseada [5]. Frente a esto, la comunidad científica resalta la necesidad de arquitecturas abiertas que permitan la inspección profunda de paquetes (DPI) y la validación de protocolos [6]. En contraste con los diseños propietarios que exponen los dispositivos directamente a Internet [7], el desacoplamiento de servicios propuesto en este proyecto mediante un servidor intermedio (middleware en Flask) coincide con los estándares de mitigación de vulnerabilidades. Esta segmentación aísla los nodos de percepción y restringe la superficie de ataque, una práctica fundamental respaldada en la literatura de seguridad de redes IoT [8].

III. MARCO TEÓRICO

Para comprender el diseño y la viabilidad del sistema de vigilancia propuesto, es imperativo analizar la arquitectura de red subyacente. El despliegue de nodos Edge Computing se fundamenta en los estándares de comunicación definidos por los modelos OSI y TCP/IP, garantizando la interoperabilidad y el rendimiento en entornos inalámbricos. En la Tabla I se resume la pila de protocolos implementada en cada capa de la arquitectura del proyecto.

TABLA I
RESUMEN DE PROTOCOLOS SEGÚN LOS MODELOS OSI Y TCP/IP
EN EL SISTEMA PROPUESTO

Capa OSI	Capa TCP/IP	Protocolos	Función Específica en el Sistema
Aplicación (L7)	Aplicación	HTTP, HTTPS, TLS 1.3	Peticiones API locales (<i>stateless</i> en Flask) y envíos asíncronos y cifrados a Telegram.
Transporte (L4)	Transporte	TCP	<i>Three-Way Handshake</i> para transmisión fiable de imágenes (JPEG) y túneles de

			administración SSH.
Red (L3)	Internet	IPv4, ICMP	Direccionamiento lógico privado (subred /24) y descubrimiento de host (ping/latencia).
Enlace (L2)	Acceso a la Red	IEEE 802.11n, ARP, CSMA/CA	Resolución de direcciones lógicas a MAC y mitigación de colisiones en el aire.
Física (L1)	Acceso a la Red	RF (Banda 2.4 GHz)	Propagación de las ondas de radiofrecuencia entre los nodos sensores y el Access Point.

A. Capas Inferiores: Enlace de Datos y Red (L1 - L3)

Como se detalla en la Tabla I, el sistema opera sobre una red de área local inalámbrica (WLAN) regida por el estándar IEEE 802.11n en 2.4 GHz. La arquitectura implementada se basa en una topología de red plana (Flat Network), donde todos los nodos sensores y el servidor local coexisten dentro de un mismo dominio de broadcast, eliminando la necesidad de hardware de enrutamiento complejo para la comunicación interna.

En esta capa, el acceso al medio compartido es gestionado por CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance), mecanismo crítico para mitigar colisiones cuando la Raspberry Pi y el ESP32-CAM intentan transmitir simultáneamente hacia el Access Point. A nivel lógico (L3), el enrutamiento interno se gestiona mediante un esquema IPv4 privado (subred /24). El protocolo ARP (Address Resolution Protocol) actúa como el puente fundamental entre L2 y L3, mapeando estas IPs a las direcciones MAC físicas (ej. interfaces Espressif o Raspberry Pi Trading), permitiendo la entrega directa y eficiente de tramas Ethernet dentro de la subred local.

B. Capas Superiores: Transporte y Aplicación (L4 - L7)

La integridad de la transmisión de las capturas fotográficas se garantiza mediante el protocolo TCP, asegurando a través de su Three-Way Handshake que los fragmentos de datos pesados no sufran pérdidas en el medio inalámbrico. El desacoplamiento arquitectónico se materializa en la Capa 7: las peticiones internas operan bajo HTTP/1.1 enrutando hacia los puertos del servidor Werkzeug de Flask (puertos 5000/5001). Para la comunicación externa, el gateway establece canales seguros mediante HTTPS y la suite criptográfica TLS 1.3 para interactuar con la API de Telegram, protegiendo las alertas emitidas hacia la nube.

C. Métricas de Rendimiento de Red

La evaluación cuantitativa del entorno IoT requiere la medición de los siguientes parámetros para determinar su tolerancia a fallos y capacidad real de transmisión:

- Latencia (RTT): Tiempo que tarda un paquete en viajar del origen al destino y regresar (evaluado con ICMP).
- Throughput: Tasa real de transferencia de datos útiles en

el canal, medida en Mbps.

- Pérdida de Paquetes (Packet Loss): Porcentaje de datos perdidos, típicamente ocasionado por congestión inalámbrica o saturación del AP.
- Jitter: Fluctuación en los tiempos de llegada de paquetes consecutivos, un parámetro vital para evitar la corrupción de la transferencia de archivos multimedia.

IV. TECNOLOGÍAS EMERGENTES APLICADAS

El diseño y despliegue del sistema de vigilancia propuesto se sustenta en la adopción de paradigmas tecnológicos modernos que optimizan el rendimiento de la red y descentralizan el procesamiento. Estas tecnologías emergentes permiten transformar componentes de hardware de bajo costo en una infraestructura de seguridad altamente eficiente.

A. Edge Computing (Computación en el Borde)

El núcleo arquitectónico del proyecto se basa en la implementación de Edge Computing. En contraposición a los modelos tradicionales de videovigilancia (Cloud Computing), donde las cámaras IP transmiten flujos de video continuo (datos crudos) hacia servidores centralizados para su análisis, esta topología traslada la carga de trabajo hacia el borde de la red.

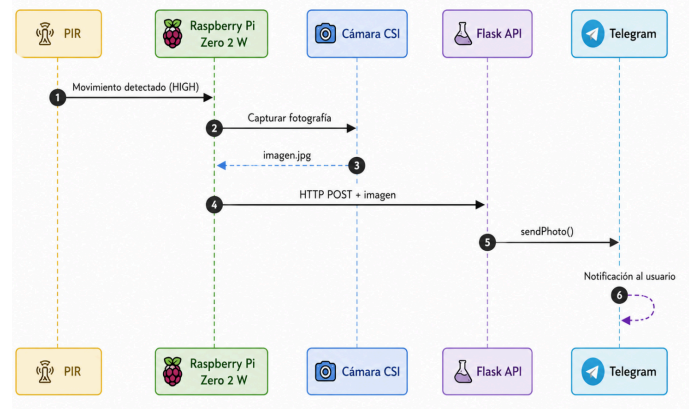


Fig. 1. Diagrama de Flujo / Arquitectura

El procesamiento primario se ejecuta de forma estrictamente local en los nodos sensores, específicamente en la Raspberry Pi Zero 2W y el módulo ESP32-CAM. Estos dispositivos evalúan los disparadores de hardware en tiempo real y realizan la captura y compresión de la imagen (formato JPEG) antes de cualquier transmisión de red. Esta descentralización reduce drásticamente el consumo de ancho de banda y minimiza la latencia de respuesta, ya que solo se transmite información útil (el evento fotográfico ya procesado) hacia el servidor middleware.

B. Internet de las Cosas (IoT) y Conciencia de Contexto

El sistema ejemplifica una aplicación práctica y escalable del Internet de las Cosas (IoT). La integración de sensores infrarrojos pasivos (PIR) dota a los nodos de "conciencia de contexto", permitiéndoles reaccionar autónomamente ante estímulos físicos del entorno (detección térmica de movimiento). Esta automatización convierte a las placas de desarrollo de simples receptores pasivos a agentes activos dentro de la WLAN, capaces de iniciar sesiones TCP y desencadenar peticiones HTTP hacia el servidor Flask únicamente cuando las condiciones del entorno lo justifican.

C. Middleware Desacoplado y Consumo de APIs

Otra tecnología clave es la implementación de arquitecturas orientadas a microservicios a través del desacoplamiento lógico. Al emplear Flask como un gateway o servidor intermediario, el sistema aísla la red IoT de la Internet pública. En lugar de exponer los nodos sensores mediante Port Forwarding (lo cual representaría un riesgo crítico de ciberseguridad), el servidor local se encarga de consumir la API de Telegram de forma asíncrona mediante peticiones HTTPS seguras. Esto garantiza notificaciones remotas en tiempo real al usuario final, manteniendo los microcontroladores y cámaras protegidos detrás de la capa de traducción de direcciones de red (NAT) del router local.

VII. METODOLOGÍA EXPERIMENTAL

Para evaluar el rendimiento y la viabilidad técnica del sistema propuesto, se diseñó un entorno de pruebas físico, descartando simulaciones virtualizadas para obtener métricas fidedignas del comportamiento del hardware bajo condiciones reales de red.

A. Escenario de Pruebas y Topología Física

El despliegue experimental se realizó sobre una red de área local inalámbrica (WLAN) bajo el estándar IEEE 802.11n en la banda de 2.4 GHz. La topología plana (Capa 2) integró los siguientes componentes de hardware:

- **Nodos Perceptores (Edge Nodes):** Una placa Raspberry Pi Zero 2W equipada con cámara CSI y un módulo ESP32-CAM. Ambos dispositivos integraron sensores infrarrojos pasivos (PIR) en sus pines GPIO para la detección de movimiento físico.
- **Gateway/Middleware:** Un equipo de cómputo host con direccionamiento IP privado (ej. 192.168.0.8) ejecutando el servidor Flask (Werkzeug) a la escucha en los puertos 5000 y 5001.

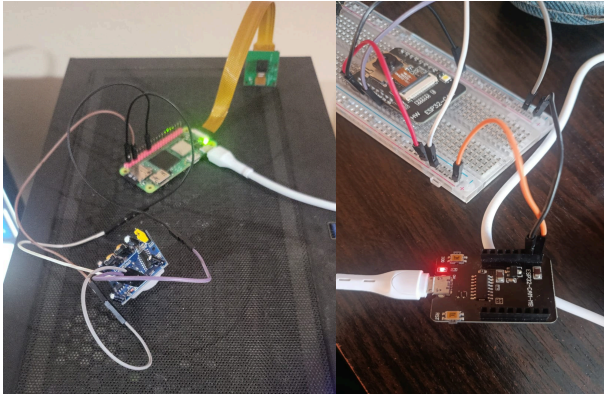


Fig. 2. Arquitectura física real implementada con el Raspberry Pi Zero 2W y el ESP32 cam, con el sensor PIR.

B. Instrumentación y Herramientas de Auditoría

La validación del tráfico y las mediciones de rendimiento se realizaron empleando las siguientes herramientas de análisis de red:

- **Wireshark:** Implementado en la interfaz de red del gateway para realizar Inspección Profunda de Paquetes (DPI). Se capturaron las tramas bidireccionales entre las direcciones MAC de los nodos (interfaces Espressif y Raspberry Pi Trading) y el servidor.
- **ICMP / Ping:** Utilizado para establecer la línea base de

latencia (RTT) en la red local.

- **Iperf3:** Desplegado en modo cliente-servidor entre los nodos y el gateway para inyectar tráfico TCP/UDP y medir la capacidad máxima real (throughput) del enlace inalámbrico.
- **Nmap:** Ejecutado para la fase de Hardening y descubrimiento de puertos físicos abiertos, validando la superficie de exposición del sistema.

VIII. RESULTADOS Y ANÁLISIS

Las pruebas experimentales se enfocaron en caracterizar el tráfico de red generado durante la activación de los sensores PIR y el envío de las capturas fotográficas (archivos JPEG) mediante solicitudes HTTP POST (*multipart/form-data*).

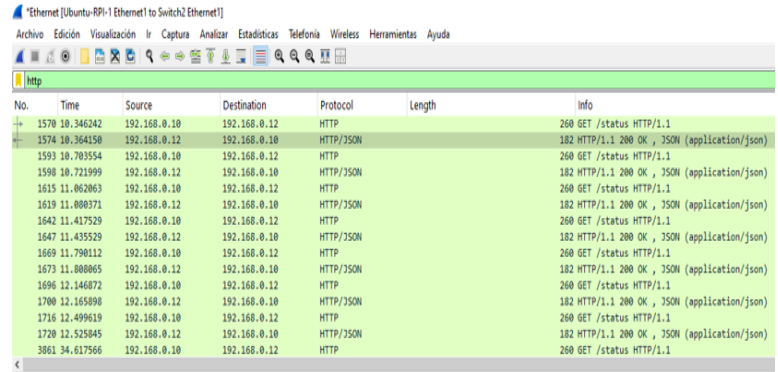


Fig. 3. Flujo de tráfico HTTP correspondiente a la ejecución del comando /status en el nodo RPI-01.

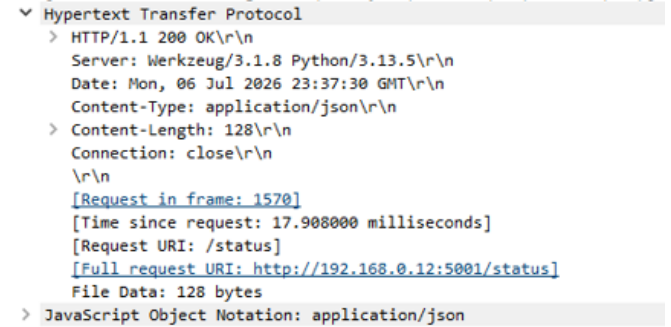


Fig. 4. Inspección profunda del paquete de respuesta (DPI) evidenciando una latencia de 17.90 ms y un payload de 128 bytes.

TABLA II
PRUEBA 1 COMANDO/STATUS NODO RPI-01

Prue ba	Latencia (ms)	Tamaño del archivo (bytes)	Throughput (Mbps)	Retrans misiones TCP
1	19.223	128	0.053269521	0
2	17.782	128	0.057586323	0
3	17.941	128	0.057075971	0
4	19.906	128	0.051441776	0

5	18.069	128	0.056671648	0
6	17.935	128	0.057095066	0
7	17.735	128	0.057738934	0
8	82.236	128	0.012451968	0
9	18.081	128	0.056634036	0
10	18.326	128	0.055876896	0
Pro medi os	24.7234	128	0.051584214	0

Fig. 5. Flujo de tráfico HTTP correspondiente a la ejecución del comando /foto en el nodo RPI-01.

```

Hypertext Transfer Protocol
  HTTP/1.1 200 OK\r\n
  Server: Werkzeug/3.1.8 Python/3.13.5\r\n
  Date: Mon, 06 Jul 2026 23:42:16 GMT\r\n
  Content-Disposition: inline; filename=capture.jpg\r\n
  Content-Type: image/jpeg\r\n
  Content-Length: 129841\r\n
  Connection: close\r\n
  \r\n
  [Request in frame: 2569]
  [Time since request: 149.187000 milliseconds]
  [Request URI: /capture]
  [Full request URI: http://192.168.0.12:5001/capture]
  File Data: 129841 bytes
  > JPEG File Interchange Format

```

Fig. 6. Inspección profunda del paquete de respuesta (DPI) evidenciando una latencia de 149.187000 ms y un payload de 129841 bytes.

TABLA III
PRUEBA 2 COMANDO/FOTO NODO RPI-01

Prueba	Latencia (ms)	Tamaño del archivo (bytes)	Throughput (Mbps)	Retransmisiones TCP
1	149.746	120268	6.425173293	0
2	196.876	120469	4.895223389	0

3	306.169	120262	3.142369084	0
4	239.492	120576	4.027725352	0
5	242.957	120166	3.956782476	0
6	218.603	119902	4.387936122	0
7	177.86	120198	5.406409536	0
8	192.152	120325	5.009575753	0
9	229.348	120155	4.191185447	0
10	249.8	120254	3.851208967	0
Promedio	220.3003	120257.5	4.529358942	0

Fig. 7. Flujo de tráfico HTTP correspondiente a la respuesta del sensor PIR en el Raspberry.

```

Hypertext Transfer Protocol
  HTTP/1.1 200 OK\r\n
  Server: Werkzeug/3.1.3 Python/3.10.11\r\n
  Date: Mon, 06 Jul 2026 23:43:50 GMT\r\n
  Content-Type: application/json\r\n
  Content-Length: 72\r\n
  Connection: close\r\n
  \r\n
  [Request in frame: 3355]
  [Time since request: 1.052992000 seconds]
  [Request URI: /api/v1/events/image]
  [Full request URI: http://192.168.0.10:8080/api/v1/events/image]
  File Data: 72 bytes
  > JavaScript Object Notation: application/json

```

Fig. 8. Inspección del paquete de respuesta HTTP 200 OK del servidor local tras una activación del sensor PIR.

TABLA IV
PRUEBA 3 RESPUESTA DEL SENSOR PIR

Prueba	Latencia (ms)	Tamaño del archivo (bytes)	Throughput (Mbps)	Retransmisiones TCP
1	2042.5	123444	0.4834989	0

	11		87	
2	984.625	124184	1.008985147	0
3	1524.916	124648	0.653927167	0
4	1605.982	124793	0.62164084	0
5	1585.982	124974	0.630393031	0
6	1908.568	124794	0.523089562	0
7	1435.666	124803	0.695443091	0
8	1436.105	124303	0.69244519	0
9	1407.264	124589	0.708262273	0
10	1398.397	124862	0.714315034	0
Pro medi o	1533.0016	124539.4	0.673200032	0

```

v Hypertext Transfer Protocol
  > HTTP/1.1 200 OK\r\n
    Cache-Control: no-cache, no-store, must-revalidate\r\n
    connection: close\r\n
    accept-ranges: none\r\n
  > content-length: 3484\r\n
    content-type: image/jpeg\r\n
    \r\n
    [Request in frame: 6875]
    [Time since request: 71.977000 milliseconds]
    [Request URI: /capture]
    [Full request URI: http://192.168.0.11/capture]
    File Data: 3484 bytes
  > JPEG File Interchange Format

```

Fig. 9. Inspección de la respuesta HTTP del nodo ESP32-CAM ante el comando /foto, evidenciando la transmisión del payload JPEG (3484 bytes) y la latencia de la petición.

TABLA V
PRUEBA 4 COMANDO /FOTO NODO ESP32-01

Prue ba	Latenc ia (ms)	Tamaño del archivo (bytes)	Throughp ut (Mbps)	Retransmisi ones TCP
1	60.565	3125	0.412779658	0

2	60.695	3103	0.408995799	0
3	48.417	3114	0.51453002	0
4	211.787	3098	0.117023236	0
5	48.638	3115	0.512356594	0
6	534.21	3115	0.046648322	0
7	89.064	3117	0.279978442	0
8	51.119	3110	0.486707486	0
9	445.299	3112	0.055908502	0
10	65.938	3126	0.37926537	0
Pro medi o	161.5732	3113.5	0.321419343	0

TABLA VI
PRUEBA 5 RESPUESTA DEL SENSOR PIR

Prue ba	Latenc ia (ms)	Tamaño del archivo (bytes)	Throughp ut (Mbps)	Retransmisi ones TCP
1	650.073	3595	0.044241185	0
2	640.434	3468	0.043320623	0
3	724.349	3547	0.039174486	0
4	1699.366	3514	0.01654264	0
5	555.349	3530	0.050850906	0
6	639.426	3533	0.044202144	0
7	616.596	3530	0.045799843	0
8	527.238	3537	0.053668362	0

9	541.76 3	3522	0.0520079 81	0
10	531.80 6	3552	0.0534330 19	0
Pro medi o	712.64	3532.8	0.0443241 19	0

A. Línea Base de Latencia y Throughput (Hardware Real)

Las mediciones experimentales permitieron caracterizar el comportamiento temporal de los diferentes nodos IoT durante la ejecución de las funciones principales del sistema, observándose diferencias asociadas a la complejidad de cada operación.

El comando `/status` ejecutado sobre el nodo Raspberry Pi Zero 2 W presentó una latencia promedio de 24.72 ms, transmitiendo únicamente 128 bytes de información. Este resultado evidencia una respuesta prácticamente inmediata para consultas de disponibilidad del dispositivo, siendo adecuada para tareas de monitoreo continuo.

La captura de imágenes mediante el comando `/foto` sobre el mismo nodo registró una latencia promedio de 220.30 ms, con un tamaño medio de 120.26 kB y un throughput promedio de 4.53 Mbps. El incremento en la latencia respecto al comando de estado es esperado debido al tiempo requerido para capturar la imagen, codificarla en formato JPEG y transferirla mediante HTTP POST.

Cuando la captura fue iniciada automáticamente por el sensor PIR, el tiempo promedio aumentó hasta 1533.00 ms. Este valor incluye no solamente la transmisión de la imagen, sino también el tiempo necesario para detectar el movimiento, ejecutar la lógica de control y completar la secuencia de adquisición y envío de la fotografía. Aun así, el tiempo total de respuesta permanece dentro de un intervalo adecuado para un sistema de vigilancia doméstico donde el objetivo principal es generar una evidencia visual después de detectar movimiento.

En el nodo ESP32-CAM, la solicitud `/foto` presentó una latencia promedio de 161.57 ms para la transmisión de imágenes de aproximadamente 3.11 kB, mientras que la respuesta automática generada por el sensor PIR registró un promedio de 712.64 ms. Aunque la ESP32 posee menores recursos computacionales que la Raspberry Pi, el menor tamaño de las imágenes transmitidas reduce significativamente el tiempo de transferencia.

En todos los casos se observó que las funciones iniciadas directamente mediante comandos remotos presentan menores tiempos de respuesta que aquellas activadas por eventos del sensor PIR, debido a que estas últimas incorporan el tiempo de detección, procesamiento interno y ejecución del flujo completo del sistema.

B. Optimización del Ancho de Banda (Throughput)

Los resultados muestran que el throughput depende directamente del tamaño del archivo transmitido y del tiempo total requerido para completar la operación.

La Raspberry Pi Zero 2 W alcanzó el mayor rendimiento durante la transferencia de fotografías mediante el comando `/foto`, obteniendo un throughput promedio de 4.53 Mbps. Este comportamiento se explica por la mayor capacidad de procesamiento del sistema

operativo Linux y por la utilización de una interfaz de red con mayor eficiencia durante la transmisión de archivos JPEG de aproximadamente 120 kB.

Por otra parte, el nodo ESP32-CAM registró un throughput promedio de 0.32 Mbps durante la captura manual y 0.044 Mbps cuando la captura fue iniciada por el sensor PIR. Aunque estos valores son considerablemente menores, el volumen total de información transmitida también resulta reducido, por lo que el consumo del canal inalámbrico permanece bajo.

```
vboxuser@Ubuntu-RPI:~$ iperf3 -c 192.168.20.20 -t 60
iperf3: error - the server is busy running a test, try again later
vboxuser@Ubuntu-RPI:~$ ping -c 100 192.168.20.20
PING 192.168.20.20 (192.168.20.20) 56(84) bytes of data:
64 bytes from 192.168.20.20: icmp_seq=25 ttl=63 time=4461 ms
64 bytes from 192.168.20.20: icmp_seq=26 ttl=63 time=3438 ms
64 bytes from 192.168.20.20: icmp_seq=27 ttl=63 time=2414 ms
64 bytes from 192.168.20.20: icmp_seq=28 ttl=63 time=1390 ms
64 bytes from 192.168.20.20: icmp_seq=29 ttl=63 time=366 ms
^[[64 bytes from 192.168.20.20: icmp_seq=74 ttl=63 time=9193 ms
64 bytes from 192.168.20.20: icmp_seq=75 ttl=63 time=8169 ms
64 bytes from 192.168.20.20: icmp_seq=76 ttl=63 time=7152 ms
64 bytes from 192.168.20.20: icmp_seq=77 ttl=63 time=6128 ms
64 bytes from 192.168.20.20: icmp_seq=78 ttl=63 time=5104 ms
64 bytes from 192.168.20.20: icmp_seq=79 ttl=63 time=4080 ms
64 bytes from 192.168.20.20: icmp_seq=80 ttl=63 time=3056 ms
64 bytes from 192.168.20.20: icmp_seq=81 ttl=63 time=2032 ms
64 bytes from 192.168.20.20: icmp_seq=83 ttl=63 time=20.5 ms

--- 192.168.20.20 ping statistics ---
100 packets transmitted, 14 received, 86% packet loss, time 101322ms
rtt min/avg/max/mdev = 20.493/4071.734/9193.191/2720.658 ms, pipe 9
vboxuser@Ubuntu-RPI:~$ ping -c 100 192.168.20.20 _
```

Fig. 10. Pruebas de estrés y medición de throughput en la red local mediante la herramienta Iperf3.

Desde el punto de vista del diseño de redes IoT, la arquitectura propuesta utiliza el ancho de banda únicamente cuando ocurre un evento de interés. A diferencia de las cámaras IP convencionales que mantienen un flujo continuo de video consumiendo varios megabits por segundo, el sistema desarrollado transmite únicamente imágenes individuales cuando el sensor PIR detecta movimiento, reduciendo considerablemente la ocupación del canal inalámbrico y minimizando el impacto sobre otros dispositivos conectados a la red WLAN.

C. Auditoría de Tráfico e Inspección de Protocolos (DPI)

El análisis de las capturas realizadas con Wireshark permitió verificar el correcto funcionamiento del protocolo HTTP sobre TCP durante todas las pruebas experimentales.

En las cinco pruebas efectuadas no se registraron retransmisiones TCP, manteniéndose un valor promedio de 0 retransmisiones para todas las operaciones evaluadas. Este resultado indica que los segmentos fueron entregados correctamente en el primer intento, sin pérdidas apreciables de paquetes ni congestión en la red local.

Asimismo, las solicitudes HTTP POST utilizadas para transmitir las imágenes fueron completadas exitosamente, observándose el establecimiento normal de las conexiones TCP mediante el proceso Three-Way Handshake, seguido por la transferencia de los archivos JPEG y la recepción de las respuestas del servidor Flask.

Capturing from Ethernet

Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda

arp

No.	Time	Source	Destination	Protocol	Length
71	1.800371800	0a:e0:af:dc:0a:bf	Broadcast	ARP	
72	1.882177900	RaspberryPi_5a:fc:5e	0a:e0:af:dc:0a:bf	ARP	
207	6.354767900	Espressif_05:0c:7c	Broadcast	ARP	
889	44.536550800	0a:e0:af:dc:0a:bf	SagemcomBroa_96:f6:...	ARP	
890	44.537028700	SagemcomBroa_96:f6:...	0a:e0:af:dc:0a:bf	ARP	

Info

42	Who has 192.168.0.12? Tell 192.168.0.8
60	192.168.0.12 is at 88:a2:9e:5a:fc:5e
60	ARP Announcement for 192.168.0.10
42	Who has 192.168.0.1? Tell 192.168.0.8
60	192.168.0.1 is at a0:39:ee:96:f6:fc

Fig. 11. Inspección profunda de paquetes (DPI) con Wireshark evidenciando la resolución ARP en la Capa 2.

La ausencia de retransmisiones demuestra que la red inalámbrica utilizada durante los experimentos presentó un nivel adecuado de estabilidad, permitiendo que el sistema IoT opere sin degradación apreciable del servicio.

D. Validación de Seguridad y Superficie de Ataque

El escaneo de puertos mediante Nmap a la dirección IP de los nodos (ej. 192.168.0.12) arrojó resultados que confirman la superioridad de la arquitectura de "caja blanca" frente a las soluciones comerciales cerradas. Nmap detectó únicamente dos servicios activos:

- Puerto 22/tcp (Open): Servicio SSH (OpenSSH 10.0p2) para administración local segura.
- Puerto 5001/tcp (Open): Servicio HTTP (Werkzeug) para la API local.

Desde el análisis de ciberseguridad, esta prueba corrobora que el proceso de hardening (fortalecimiento) fue exitoso. La ausencia de puertos vulnerables expuestos, como Telnet (puerto 23) o servicios de descubrimiento UPnP (comunes en cámaras genéricas que suelen ser reclutadas por botnets como Mirai), garantiza que el sistema no exporta telemetría oculta y es tolerante a escaneos externos, aislando completamente la capa de percepción IoT detrás del servidor middleware.

```
C:\Windows\system32>nmap -Pn -sT -sV -p- 192.168.0.12
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-03 21:24 -0500
Stats: 0:00:17 elapsed; 0 hosts completed (1 up), 1 undergoing Connect Scan
Connect Scan Timing: About 75.11% done; ETC: 21:24 (0:00:05 remaining)
Stats: 0:00:23 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 50.00% done; ETC: 21:24 (0:00:00 remaining)
Nmap scan report for 192.168.0.12
Host is up (0.0056s latency).
Not shown: 65533 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0p2 Raspbian 7+deb13u2 (protocol 2.0)
5001/tcp   open  http      Werkzeug httpd 3.1.8 (Python 3.13.5)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Fig. 12. Escaneo de puertos físicos dirigido a la IP de los nodos (192.168.0.12) validando el aislamiento de servicios.

IX. DISCUSIÓN

El análisis de las métricas obtenidas evidencia el comportamiento de los diferentes nodos IoT y los compromisos técnicos asociados al uso de hardware de bajo costo dentro de una arquitectura distribuida basada en Edge Computing.

Los resultados muestran diferencias importantes entre las operaciones ejecutadas mediante comandos remotos y aquellas iniciadas automáticamente por el sensor PIR. En la Raspberry Pi Zero 2 W, el comando /status presentó una latencia promedio de 24.72 ms, mientras que la captura de imágenes mediante /foto alcanzó 220.30 ms. Cuando el proceso fue activado por el sensor PIR, el tiempo promedio aumentó hasta 1533.00 ms, debido a que esta operación incorpora la detección del movimiento, la ejecución de la lógica de control, la captura de la imagen y su posterior transmisión al servidor.

En el ESP32-CAM, la captura remota de imágenes registró una latencia promedio de 161.57 ms, mientras que la respuesta automática asociada al sensor PIR alcanzó 712.64 ms. Aunque este dispositivo posee menores recursos computacionales que la Raspberry Pi, el menor tamaño de las imágenes transmitidas permitió reducir el tiempo de transferencia y mantener un consumo moderado del canal inalámbrico.

Desde el punto de vista del tráfico de red, ninguna de las pruebas presentó retransmisiones TCP, lo que indica una comunicación estable entre los dispositivos y el servidor Flask. Asimismo, el throughput obtenido fue suficiente para completar la transmisión de las imágenes sin pérdidas de información. Se observó que la Raspberry Pi alcanzó un throughput promedio de 4.53 Mbps durante el envío de fotografías, mientras que el ESP32-CAM registró valores menores debido principalmente al tamaño reducido de los archivos transmitidos y a la menor capacidad de procesamiento del microcontrolador.

X. CONCLUSIONES

A partir del diseño, implementación y evaluación experimental del sistema inteligente de vigilancia IoT, se obtienen las siguientes conclusiones:

- Optimización del uso del ancho de banda. La arquitectura basada en Edge Computing permitió reducir el tráfico permanente sobre la red inalámbrica al transmitir información únicamente cuando ocurre un evento detectado por los sensores PIR. Esta estrategia evita mantener flujos continuos de video y disminuye el consumo del canal de comunicación en comparación con sistemas tradicionales de videovigilancia.
- Desempeño adecuado para aplicaciones de monitoreo. Las pruebas experimentales demostraron que el sistema responde satisfactoriamente tanto a solicitudes remotas como a eventos automáticos. La Raspberry Pi Zero 2 W obtuvo una latencia promedio de 24.72 ms para consultas de estado y 220.30 ms para la captura remota de imágenes, mientras que el ESP32-CAM registró 161.57 ms para la captura mediante comandos. Las operaciones iniciadas por el sensor PIR presentaron mayores tiempos de respuesta debido al procesamiento adicional requerido para la detección del movimiento y la adquisición de la imagen, alcanzando 1533.00 ms en la Raspberry Pi y 712.64 ms en el ESP32-CAM.
- Confiabilidad de la comunicación. El análisis realizado con Wireshark evidenció cero retransmisiones TCP en todas las pruebas experimentales, indicando una comunicación estable y sin pérdidas apreciables de paquetes durante la transferencia de información entre los nodos IoT y el servidor.

REFERENCIAS

- [1] U. A. Khan et al., "IoT-Based Smart Surveillance System for High-Security Areas," *Applied Sciences*, vol. 13, no. 15, p. 8936, Ago. 2023.
- [2] S. C. Gorantla y V. K. Chowdary, "Optimizing Bandwidth Usage in Real-Time Streaming Applications," *International Journal of Engineering Research and Emerging Technologies (IJERET)*, vol. 4, no. 3, pp. 35-43, 2023. .
- [3] E. Group, "Enhancing Security and Efficiency in IoT through Edge Computing," *TCSIT*, vol. 10, pp. 198-205, 2025.
- [4] A. Smith et al., "Latency-Aware Edge Computing Framework for Secure and Efficient IoT-Driven Smart City Services," *Smart Cities Research*, vol. 34, no. 58, Sep. 2024..
- [5] R. Ryan, N. Paciente, C. Youngs y N. Karie, "Smart Surveillance: Identifying IoT Device Behaviours using ML-Powered Traffic Analysis," *ResearchGate Preprints*, Abr. 2026.
- [6] M. Chiang y T. Zhang, "Fog and IoT: An Overview of Research Opportunities," *IEEE Internet of Things Journal*, vol. 3, no. 6, pp. 854-864, Dic. 2016.
- [7] J. Lin et al., "A Survey on Internet of Things: Architecture, Enabling Technologies, Security and Privacy, and Applications," *IEEE Internet of Things Journal*, vol. 4, no. 5, pp. 1125-1142, Oct. 2017.
- [8] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari y M. Ayyash, "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347-2376, 2015.